

Qué es: un caso de cierre de la unidad de dimensionamiento. Usa lo de las tres clases de Dimensionamiento de servidores (de bytes a Mbps, HTML contra JSON, nube contra servidor propio), el simulador de capacidad (workers, latencia alta contra fallas directas, RNF de capacidad) y los casos fallidos (Healthcare.gov, Prime Day). La parte de seguridad es introductoria: se entiende con la idea de puerto abierto y leyendo un log. Los estudiantes leen el caso en el celular (pablopedrerna0.github.io/caso-siga-af/) y completan la planilla impresa en grupos de 2 o 3.

Para copiar en el pizarrón

Lunes	07:50	08:15	14:00
Pedidos/s que llegan	4	180	9
Pedidos/s atendidos	4	22	9
Latencia p95	0,3 s	14 s	0,4 s
CPU del servidor	12 %	28 %	14 %
Conexiones a la base	3	3	3
Enlace (de 100)	30 Mbps	74 Mbps	33 Mbps

El enlace incluye el resto del edificio: unos 30 Mbps en el pico. La tasa de error no va en el pizarrón porque la calculan ellos en la pregunta 1.

Tiempos (60 min)

- **5'**: pizarrón, consigna, armar grupos y pasar el link.
- **10'**: lectura del caso.
- **15'**: partes 1 y 2 (diagnóstico y cuenta).
- **20'**: partes 3 y 4 (seguridad y propuesta).
- **10'**: puesta en común. Que dos grupos que eligieron opciones distintas en la pregunta 10 (nube o enlace) las defiendan.

Si falta tiempo, la 11, la 12 y la 13 se pueden terminar en casa. Las partes 1 a 3 tienen respuesta cerrada; la parte 4 es abierta y se evalúa por la justificación.

Respuestas esperadas

1. Llegan 180 y se atienden 22: $(180 - 22) / 180 \approx 88 \%$ sin atender.
2. **Los workers de la app.** Los atendidos quedan clavados en 22 mientras llegan 180, y el p95 sube a 14 s porque se arma una cola. No era la CPU (28 %, lejos del 100), tampoco la base (siempre 3 conexiones, estaba tranquila) ni el enlace (74 de 100, no estaba lleno).
3. La app tiene **1 solo worker**: atiende **un pedido a la vez**. Es el síntoma de "latencia alta" del simulador: faltan procesos, no memoria. Con 4 vCPU, un solo proceso usa como mucho un núcleo, o sea el 25 %. Encima pasa la mayor parte del tiempo esperando a MySQL, así que ni siquiera ese núcleo se llena. **La máquina estaba ociosa: comprar hardware no lo arreglaba.**
4. $1 \div 0,045 \approx 22$ **pedidos/s**, que coincide con el pizarrón. Es el techo de un worker que atiende de a uno.
5. **Empeoraba.** Cada F5 es un pedido nuevo que se suma a la cola, y además vuelve a bajar la foto de 200 KB. Es lo mismo que pasó con Healthcare.gov en los casos fallidos.
6. $180 \times 250 = 45.000 \text{ KB/s} = 45 \text{ MB/s} \rightarrow 360 \text{ Mbps}$, + 30 = **390 Mbps**. No alcanzan ni 100 ni 300. **Si arreglan solo la app, el cuello de botella pasa al enlace.**
7. $180 \times 30 = 5.400 \text{ KB/s} \rightarrow 43,2 \text{ Mbps} + 30 \approx 73 \text{ Mbps}$: **entra en 100**. Conclusión: achicar lo que se manda (caché de imágenes y estilos) no cuesta nada y rinde más que pagar un enlace más grande. Conecta con la clase de HTML contra JSON.
8. **3306**: cualquiera desde internet puede probar la base, y con la contraseña del ejemplo (que es pública) entra como root y lee, cambia o borra todo. Para cerrarlo: cerrar el 3306 a internet (que solo la app llegue a la base), poner una contraseña fuerte y que Secretaría entre por VPN. **8080**: es el panel de administración de la base, abierto a internet. Hay que cerrarlo a internet y dejarlo accesible solo desde la red interna o por VPN. **Domingo**: un ataque de fuerza bruta (probar contraseñas una tras otra), y **funcionó**: el 302 seguido de GET /panel es un login correcto, o sea que

alguien entró como admin la noche anterior a la inscripción. Esto es un incidente: hay que cambiar la contraseña ya, revisar en los logs qué hizo en /panel y si tocó cupos o inscripciones, y limitar los intentos (bloquear la IP o la cuenta después de N fallos). Si nadie se da cuenta de que el login funcionó, marcarlo en la puesta en común.

9. **Arquitectura (9):** internet → servidor web (80/443, lo único abierto a internet) → app con varios workers → base de datos, cerrada a internet. Un monitoreo que mida pedidos/s, latencia, errores, CPU y enlace, con su panel no expuesto o detrás de un login. Si nombran Prometheus/Grafana, bien, pero no hace falta.
10. **Decisiones (10):** $180 \div 22 \approx 8,2$, así que **9 workers**. Si alguien conoce la regla habitual de $2 \times \text{núcleos} + 1$, también da 9 con 4 núcleos. Es lo mismo que hicieron con `sumar_cpu.sh` en el simulador, pero con una cuenta. **Nube o enlace:** las dos opciones se pueden defender. Si aplican el caché, el enlace de 100 alcanza y la plata puede ir a la nube por 48 h: más CPU para los workers, un enlace de 1 Gbps y SIGA deja de competir con el wifi del edificio (el riesgo es migrar la base antes y traerla de vuelta después). El enlace de 300 le sirve al edificio todo el año, pero **sin caché no alcanza** (390). Si eligen sin dar un número, no cuenta. **Peso:** caché de imágenes y estilos en el servidor web, comprimir o achicar la foto, compresión (gzip) como en la clase 3. Si alguien propone mandar JSON y armar la página en el navegador, conectarlo con HTML contra JSON.
11. **Alertas (11), por ejemplo:** $p95 > 1$ s durante 2 min; errores $5xx > 1$ %; CPU > 80 %; enlace > 80 Mbps; conexiones a la base cerca del máximo; más de 20 logins fallidos por minuto desde una misma IP. Lo importante es que la tercera columna diga **qué hace** el administrador cuando salta.
12. **RNF (12), por ejemplo:** 200 pedidos/s (180 más un margen), $p95 < 1$ s, errores < 1 %. Si se supera, una página liviana de "sala de espera" o una portada reducida, como el fallback de Prime Day.
13. **Prueba (13):** una prueba de carga contra una copia del sistema (no contra el real): mandarle muchos pedidos simultáneos, como el `subir_carga.sh` del simulador, e ir subiendo hasta 200 pedidos/s mientras miran latencia, errores y uso del enlace. Medir el tiempo de respuesta con `curl` como en la práctica de Killercoda. Herramientas reales: ab, JMeter, k6.